DOCUMENTO OFICIAL DE PREPARACIÓN

Guía Maestra y Banco de Preguntas para la Defensa Oral

Segundo Entregable - Lenguajes de Programación II (TUDS - Universidad Católica de Córdoba)

Integrantes: Ignacio Ortiz | Leonardo Manrique | Luis Poblete

1. Resumen Técnico del Proyecto

El sistema es una API RESTful desarrollada con **Spring Boot 3.2.5**, **Spring Security 6** y **Spring Data JPA** para la gestión de Usuarios y Reservas Gastronómicas. Implementa arquitectura limpia en capas desacopladas (Controller, Service, Repository, Entity, DTO y Mapper), persistencia relacional con H2 y MySQL, autenticación HTTP Basic stateless y control de accesos basado en roles (RBAC).

Componente	Tecnología / Implementación	Responsabilidad Principal
Framework & Java	Java 21 LTS / Spring Boot 3.2.5	Núcleo de la API REST y contenedor de inyección de dependencias.
Seguridad	Spring Security + Basic Auth	Filtro de peticiones HTTP, autenticación stateless y RBAC.
Cifrado	BCryptPasswordEncoder (Salt)	Hash unidireccional de contraseñas almacenadas en la base de datos.
Persistencia	Spring Data JPA + Hibernate	Mapeo relacional de entidades, repositorios y consulta por relación.
Transferencia	DTOs + Mappers dedicados	Aislamiento del modelo de datos para evitar exposición de credenciales.
Validaciones	Jakarta Bean Validation (@Valid)	Validación de parámetros de entrada antes de alcanzar el servicio.

2. Conceptos Teóricos Fundamentales (Preguntas Obligatorias)

> **Criterio Crucial de la Cátedra:** Cada integrante del grupo debe saber explicar con total soltura la diferencia entre *Autenticación* y *Autorización*, y señalar en qué partes exactas del código fuente ocurre cada una.

Dimensión	Autenticación (Authentication)	Autorización (Authorization)
Pregunta Clave	**¿Quién eres?** (Verificación de identidad).	**¿Qué tienes permitido hacer?** (Permisos sobre recursos).
Implementación	`CustomUserDetailsService` + `BCryptPasswordEncoder`.	`SecurityConfig` con `.hasRole("ADMIN")`.
Código HTTP al fallar	`401 Unauthorized` (credenciales incorrectas/ausentes).	`403 Forbidden` (usuario válido pero sin rol suficiente).

Basic Auth y Arquitectura Stateless

¿Qué significa Stateless? La aplicación no almacena sesiones en memoria del servidor ni utiliza cookies (`JSESSIONID`). Se configuró mediante `sessionCreationPolicy(SessionCreationPolicy.STATELESS)`. Cada petición HTTP es autónoma y debe incluir sus credenciales en la cabecera `Authorization: Basic <base64>`.

¿Base64 es cifrado? NO. Base64 es una simple codificación reversible. Por eso, en entornos reales de producción es mandatorio utilizar **HTTPS (TLS/SSL)** para cifrar el canal de comunicación completo.

Hashing vs. Cifrado (BCrypt y Salt)

Diferencia: El cifrado es bidireccional (reversible con clave). El **hash** es una función matemática **unidireccional** (irreversible). No se puede reconstruir la contraseña original a partir del hash.

¿Cómo valida Spring Security la contraseña? Spring Security toma la contraseña enviada en la petición, le aplica la función BCrypt con el *salt* guardado en el hash existente, y compara ambos hashes con `passwordEncoder.matches(plana, hash)`.

¿Qué es el Salt? Una secuencia pseudoaleatoria que BCrypt genera automáticamente e inserta en el hash (`$2a$10$...`). Asegura que si dos usuarios eligen la misma contraseña, sus hashes en base de datos sean completamente distintos, neutralizando ataques de tablas precalculadas (Rainbow Tables).

3. Flujo Paso a Paso de una Petición Protegida

1. El cliente envía la petición (ej: `GET /reservas`) con cabecera `Authorization: Basic <usuario:clave_en_base64>`.
2. El filtro `BasicAuthenticationFilter` intercepta la petición y extrae las credenciales.
3. Invoca a `AuthenticationManager`, que delega en `CustomUserDetailsService.loadUserByUsername()`.
4. Se busca el usuario por correo en `UsuarioRepository` y se obtiene su contraseña hasheada y rol.
5. `BCryptPasswordEncoder.matches()` compara la contraseña provista contra el hash guardado.
6. Si coincide, se establece el objeto `Authentication` en `SecurityContextHolder` (identidad autenticada).
7. El interceptor de autorización contrasta los roles contra las reglas de `SecurityConfig`. Si cumple, pasa al Controller; si no, retorna `403 Forbidden`.

4. Banco Extenso de Preguntas y Respuestas para la Defensa

Bloque A: Seguridad y Autenticación

P1 ¿Por qué implementaron UserDetailsService y no usaron usuarios en memoria?

Porque en un sistema productivo los usuarios deben persistir en base de datos. Con `CustomUserDetailsService` buscamos al usuario por correo dinámicamente en `UsuarioRepository`, permitiendo que cualquier nuevo usuario registrado a través de la API pública pueda iniciar sesión de inmediato sin requerir reinicios.

P2 ¿Cómo asocia Spring Security el rol del usuario con las reglas de acceso?

En `CustomUserDetailsService` convertimos el enum `Rol` en una autoridad con el prefijo estándar `ROLE_` (`new SimpleGrantedAuthority("ROLE_" + rol.name())`). Luego, en `SecurityConfig`, al declarar `.hasRole("ADMIN")`, Spring Security comprueba automáticamente la presencia de la autoridad `ROLE_ADMIN`.

P3 ¿Por qué deshabilitaron CSRF (csrf.disable())?

CSRF aplica a aplicaciones web basadas en navegador que mantienen sesiones mediante cookies. Al diseñar una API REST con arquitectura **Stateless** y autenticación HTTP Basic donde cada petición envía credenciales en cabeceras y no almacena cookies, el riesgo de CSRF no existe y mantenerlo habilitado bloquearía innecesariamente las peticiones POST, PUT y DELETE.

P4 ¿Qué sucede si un usuario envía la contraseña ya hasheada en el Basic Auth de Postman?

La autenticación fallará invariablemente con **401 Unauthorized**. Basic Auth espera la contraseña original en texto plano. Si se envía el hash, el motor de seguridad volverá a hashear esa cadena, obteniendo un valor completamente disímil al almacenado.

P5 ¿Por qué se configuró headers.frameOptions().disable()?

Por motivos de desarrollo local: la consola web de H2 (`/h2-console`) está construida con frames HTML. Spring Security por defecto añade la cabecera de seguridad `X-Frame-Options: DENY` para prevenir ataques de Clickjacking. Deshabilitarlo para los frames permite usar la consola web para inspeccionar la base de datos visualmente.

Bloque B: Control de Acceso por Roles (RBAC)

P6 ¿Qué operación del dominio restringieron y a qué rol específico?

Restringimos la eliminación física de recursos (`DELETE /usuarios/**` y `DELETE /reservas/**`) exclusivamente al rol **ADMIN**. Si un usuario autenticado con rol estándar **USER** intenta invocar un método DELETE, el sistema deniega el acceso respondiendo **403 Forbidden**.

P7 ¿Cómo se evita que un usuario se auto-asigne el rol ADMIN al registrarse?

A través de la separación estricta en el DTO de entrada: `UsuarioRegistroDTO` no contiene la propiedad `rol` . Asimismo, en `UsuarioService.registrar()` y `UsuarioMapper` se fija programáticamente `Rol.USER` , impidiendo cualquier escalada de privilegios desde peticiones públicas.

P8 ¿Cómo crearon el administrador de prueba si el registro público no lo permite?

Creamos el componente `DataInitializer` (que implementa `CommandLineRunner`). Al iniciar la aplicación, comprueba si existe el usuario `admin@ucc.edu.ar` ; si no está, lo inserta con rol `Rol.ADMIN` y su contraseña cifrada con BCrypt (`Admin123!`).

Bloque C: Arquitectura, DTOs y Mappers

P9 ¿Por qué no devuelven directamente las entidades JPA en los Controllers?

1. **Fuga de Información (Data Leakage):** Exponer la entidad `Usuario` enviaría el hash de la contraseña en las respuestas JSON.
2. **Bucle infinito de serialización:** Las relaciones JPA bidireccionales provocan recursión infinita en Jackson (`StackOverflowError`).
3. **Desacoplamiento:** Permite modificar la base de datos sin alterar el contrato público consumido por el cliente.

P10 ¿Cómo aseguraron que la contraseña no se exponga en consultas por relación?

El DTO `ReservaResponseDTO` incluye un `UsuarioResponseDTO` anidado, el cual únicamente proyecta `id` , `nombre` , `correo` , `edad` y `rol` . El atributo contraseña no existe en el DTO. Adicionalmente, la entidad `Usuario` tiene `@JsonIgnore` en su campo password como medida de defensa en profundidad.

Bloque D: Persistencia y Relaciones JPA

P11 Expliquen cómo modelaron la relación entre Usuario y Reserva.

Es una relación muchos-a-uno real (`@ManyToOne`) en la entidad `Reserva` con la columna de unión `usuario_id` (`@JoinColumn(nullable = false)`). Cada reserva pertenece a un único usuario del sistema.

P12 ¿Cómo implementaron la consulta requerida que utiliza esa relación?

En `ReservaRepository` definimos el método de consulta derivada `findByUsuario_Id(Long usuarioId)` . Spring Data JPA genera la consulta filtrando por la foreign key. Se expone en el endpoint `GET /reservas/usuario/{usuarioId}` , verificando previamente la existencia del usuario para devolver 404 si no existe.

Bloque E: Validaciones y Códigos de Estado HTTP

P13 ¿Cómo controlan el registro de correos duplicados y qué código HTTP devuelven?

Mediante doble barrera: comprobación lógica en `UsuarioService` con `existsByCorreo()` lanzando `CorreoYaExisteException` , y restricción física en base de datos con `@Column(unique = true)` . `GlobalExceptionHandler` intercepta ambas y retorna **409 Conflict** con mensaje claro.

P14 ¿Qué diferencia hay entre el código 400 y el código 404 en su API?

400 Bad Request: Error en los datos enviados en la petición (ej: contraseña menor a 6 caracteres, correo sin formato válido, edad negativa).
404 Not Found: La sintaxis es válida pero el identificador del recurso no existe en la base de datos (ej: usuario o reserva inexistente).

Bloque F: Preguntas Desafiantes / "Trampa"

P15 ¿Qué riesgo existe si la relación Reserva -> Usuario se define con FetchType.LAZY?

Si se accede a las propiedades del usuario fuera de la sesión transaccional de Hibernate, se produce una LazyInitializationException. En nuestro caso, como la conversión de DTOs se realiza dentro de los métodos anotados con @Transactional en la capa Service, la sesión permanece abierta y se ejecuta de manera segura.

P16 **¿Por qué se limpian los correos con .trim().toLowerCase()?**

Para normalizar el identificador. En los correos electrónicos la identidad es case-insensitive: `User@Ucc.edu.ar` y `user@ucc.edu.ar` corresponden a la misma casilla. Normalizar evita duplicados por variaciones de mayúsculas y espacios accidentales.

5. Hoja de Ruta para la Demostración en Vivo (Postman Live Demo)

Para la defensa práctica en vivo ante la mesa examinadora, ejecuten las peticiones en el siguiente orden secuencial:

Paso	Acción / Petición	Método y Ruta	Auth / Credenciales	Resultado Esperado	Qué Explicar a los Profesores
1	Registro válido	`POST /auth/registro`	Público (Sin auth)	`201 Created`	Mostrar que el JSON de respuesta tiene rol USER y **no incluye password ni hash**.
2	Correo duplicado	`POST /auth/registro`	Público (Sin auth)	`409 Conflict`	Reenviar la misma petición anterior. Explicar la comprobación en service y la restricción única en base de datos.
3	Datos inválidos	`POST /auth/registro`	Público (Sin auth)	`400 Bad Request`	Enviar clave corta (<6 caracteres) y email sin '@'. Mostrar el detalle de errores validado por @Valid.
4	Ruta protegida sin auth	`GET /reservas`	Sin cabecera Auth	`401 Unauthorized`	Demostrar que las rutas del dominio están resguardadas por el filtro de Spring Security.
5	Acceso autenticado USER	`GET /reservas`	Basic: user@ucc.edu.ar / User123!	`200 OK`	Mostrar que con credenciales válidas se accede y se obtienen las reservas en formato JSON.
6	Operación restringida USER	`DELETE /usuarios/1`	Basic: user@ucc.edu.ar / User123!	`403 Forbidden`	**Punto clave:** Explicar la diferencia entre 401 (no autenticado) y 403 (autenticado pero sin rol suficiente).
7	Operación permitida ADMIN	`DELETE /reservas/1`	Basic: admin@ucc.edu.ar / Admin123!	`204 No Content`	Demostrar que el rol ADMIN sí posee el privilegio para ejecutar el borrado de entidades.
8	Consulta por relación JPA	`GET /reservas/usuario/2`	Basic: user@ucc.edu.ar / User123!	`200 OK`	Comprobar la relación JPA Reserva $\to$ Usuario y ratificar que el usuario anidado tampoco expone contraseñas.

> **Verificación visual en Base de Datos:** Abrir en el navegador `http://localhost:8080/h2-console` (JDBC URL: `jdbc:h2:file:./data/reservas_db`, Usuario: `sa`) y ejecutar `SELECT * FROM USUARIOS;` para mostrar que las contraseñas están almacenadas únicamente como hash BCrypt (`$2a$10$...`).